

DESIGN AND UNIFIED MODELING LANGUAGE

Unified Modeling Language in Domain-Driven Design

Unified Modeling Language (UML): A standardized visual notation that uses diagrams to model software systems and their components.

Domain-Driven Design (DDD): A software engineering approach that aims to align software models with how businesses work.

In DDD, the business environment is abstracted into a collection of **conceptual classes** that each represent a real-world entity.

UML is particularly useful for DDD.

Object-Oriented Analysis and Design

Object-Oriented Analysis and Design (OOAD): The process of applying object-oriented programming (OOP) concepts to the analysis and design of a system.

OOAD is commonly implemented during either of these processes:

- **Reverse engineering:** Documents the system after it is built, and distills its complex mechanisms into a comprehensible model.
- **Forward engineering:** Occurs prior to and throughout a system's stages of development.

OOAD involves gathering user stories and identifying needs, then determining how to build a system to meet those needs.

UML Diagram Types

Static modeling: Shows relationships between classes.

Dynamic modeling: Shows how information moves between classes, given a particular event.

Class-responsibility-collaboration (CRC) card: Lists a class's traits and behaviors on the left and any collaborator classes on the right.

A CRC card is a halfway point between a notebook sketch and a formal UML diagram.

In UML, a feature’s access level is called its **visibility**, which can be either public or private.

- Public features are accessible inside and outside the defined class, and they’re preceded in UML diagrams by (+).
- Private features are only accessible inside the defined class, and they’re preceded in UML diagrams by (-).

Class Diagrams and Relationships

Class diagrams: Static structure diagrams that show classes, their features, and their relationships.

There are two types of class diagrams.

1. **Domain model:** Represents a business domain.
2. **Design model:** More closely related to the desired software product.

Generalization: A relationship wherein child classes inherit attributes and methods from a parent class.

Specialization: A relationship wherein a child class only lists features that are unique to that class.

Association: A relationship wherein objects of one class contain instances of another.

There are two types of association:

1. In an **aggregation**, **Object1** has **Object2** as an attribute; **Object2** can exist independently of **Object1**.
2. In a **composition**, **Object1** owns **Object2**; **Object2** has no meaning without **Object1** and can’t exist independently.

Multiplicity: The number of instances involved in an association.

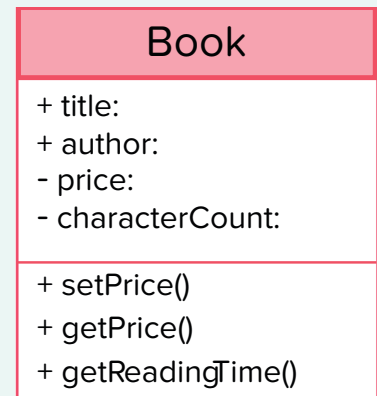
Dependency: A relationship wherein one class depends on the input of processes from another.

Keywords: Used to indicate a specific use of a class or other entity within a model.

Interface: A keyword used to specify that classes that implement an interface must provide definitions for the interface’s features.

UML is primarily used for design models.

Class Diagram



This class diagram shows a **Book** object with its public and private features.

In UML diagrams, keywords are represented within guillemets <>> above the entity’s name.

Constraints: Indicate restrictions, limits, or ranges in a class diagram that are applicable to class features.

Instance specifications: Specific instances of classes rendered in UML.

Responsibility-Driven Design and Interaction Diagrams

Responsibility-Driven Design (RDD): Focuses on defining the responsibilities of each object and then modeling the actions and behaviors needed to fulfill those responsibilities.

RDD relies on dynamic **interaction diagrams** to show the interactions between objects in a system over time.

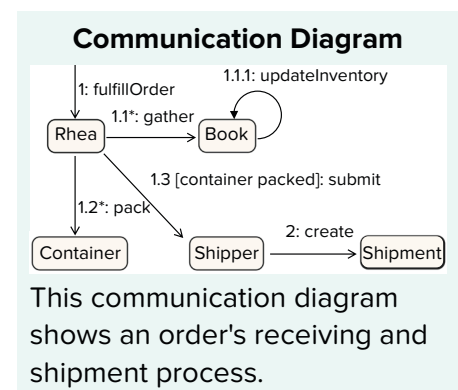
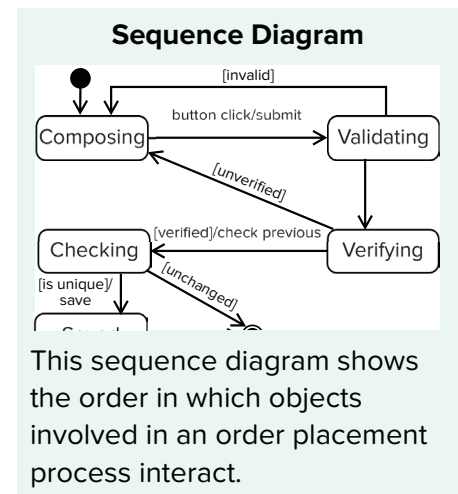
Two common types of interaction diagrams are:

1. **Sequence diagram:** Shows the order in which a system's objects interact with each other during a single process.

Parts of a Sequence Diagram	
Lifeline	A dashed line that extends downward from an object
Activation bar	A beige rectangle that shows when its respective object is active in the sequence
Message	A communication that causes some action to be taken
Gate	A message that originates from or ends up outside the sequence
Combined fragment	A group of behaviors that accomplish some task within a larger process
Operator	Defines the purpose of each combined fragment

2. **Communication diagram:** Shows the order in which a system's objects interact, with more flexibility than a sequence diagram.

- **Guard clause:** A condition that must be satisfied before some behavior can happen.



Use Cases, Use-Case Models, and Use-Case Diagrams

Use case: A narrative description of how actors—i.e., the agents or people involved—will interact with a given system.

Preconditions: System prerequisites that must be specified before the use case unfolds.

Trigger: An action that initiates the scenario defined in a use case.

Postconditions: Guarantees whose fulfillment mark the end of a scenario.

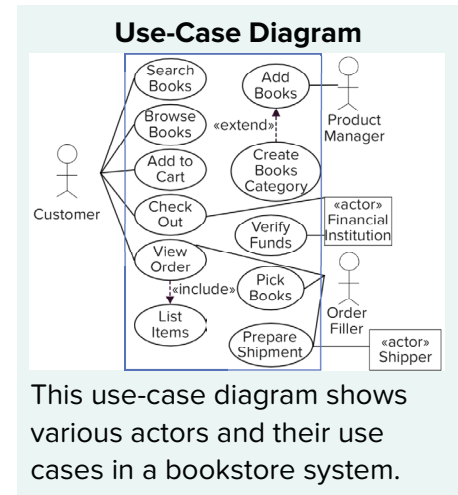
- **Minimum guarantees** are true regardless of how a scenario unfolds.
- **Success guarantees** are only true when the scenario runs successfully.

Use-case model: Includes all use cases and describes the total functionality of the system.

Use-case diagram: A visual representation of a use-case model.

Besides use cases and actors, use-case diagrams include:

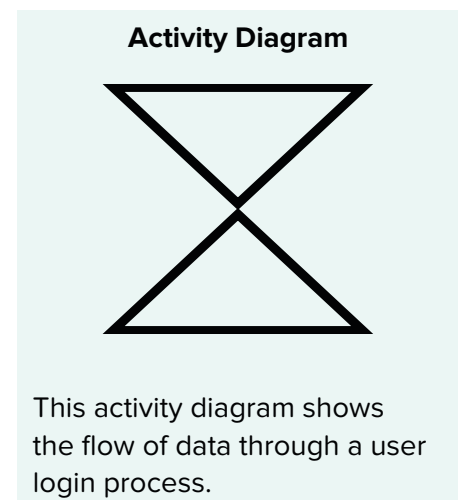
- **Subject boundaries:** Solid-line boxes containing the system.
- Indicators of any special relationships between use cases, such as an:
 1. **Extend relationship:** Indicates a use case that may be needed or not, depending on the operation of another use case.
 2. **Include relationship:** Indicates that one use case includes the functionality of another.



Activity Diagrams

Activity diagram: Shows the sequential flow of data through a system.

Activity: An event that occurs as part of a business process.



Control flows: Solid arrows showing progress through the process.

Incoming edge: A control flow pointing toward a diagram element.

Outgoing edge: A control flow pointing away from an element.

More complex logic in an activity diagram is represented by **control nodes**, or decision/action points at which subsequent process flows are determined.

Types of Control Nodes	
Initial nodes	Represent the beginning of a process
Decision nodes	Represent conditional logic
Merge nodes	Represent the joining of flows separated by decision nodes
Fork nodes	Represent the branching of a flow into multiple flows that run in parallel
Join nodes	Represent the merging of branched flows
Final-activity nodes	Represent the last activity in a process

Swimlanes: Divide actions based on actors or business units.

Object nodes: Represent objects acted on by activities.

Elements Used to Represent Trigger Activities

Time Signal 	Represents a meaningful passage of time that influences the flow
Send Signal 	Often accompanies the time signal and denotes a signal sent to an external source
Accept Signal 	Represents a signal received from an external source

State and State-Machine Diagrams

State: The condition of an object at a specific time, which is the sum total of all its attribute values.

State machine: An object that has both a state and instructions for changing its state in response to inputs.

State-machine diagram: Depicts an object's state changes and the actions that cause those changes during a given process.

Do-activity: A brief description of an object's current activity that may accompany states in a state-machine diagram.

Source state: The initial state of an object.

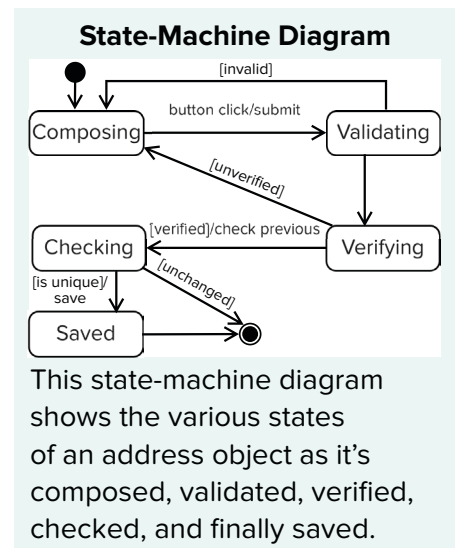
Target state: The state brought on by external input or influence.

Transitions from source to target state often include text notation for context:

- **Trigger:** An event that initiates a transition.
- **Guard:** An optional Boolean precondition that governs whether a transition can occur.
- **Behavior:** Describes how the object interacts during a transition.

Pseudostate: An object's state immediately prior to the process modeled by a state-machine diagram.

Final state: An object's state following the conclusion of the modeled process.



THE OBJECT-ORIENTED PROGRAMMING PARADIGM

Object-Oriented Programming

Object-oriented programming (OOP): A programming paradigm that models a system as a collection of objects that interact with each other. Data and the operations that apply to that data are combined in the objects.

Characteristics of OOP languages are:

- **Abstraction:** Representing an element of a real-world problem as a class with just the attributes and methods needed to accomplish a task.
- **Inheritance:** Defining a subclass that inherits the attributes and methods of a superclass.
- **Polymorphism:** Overriding inherited methods in subclasses to change their behavior.
- **Encapsulation:** Keeping an object's internal data private and allowing external access through a public interface.

Types in OOP

Objects instantiated from a subclass are of that subclass's type and of the superclass's type.

Classes from which you can directly instantiate objects are called **concrete classes**.

Classes that are only meant to be superclasses and cannot themselves be instantiated from are called **abstract classes**.

An **interface** defines a type, and classes that implement the interface must implement whatever attributes and methods the interface defines.

Important: Selecting the right combination of interfaces and concrete and abstract classes makes the semantics of your program precise.

	can be instantiated	can contain implementation
concrete class	yes	yes
abstract class	no	yes
interface	no	no

OOP Principles

The aim of good OO design is twofold:

- High **cohesion** within each class, or a focused set of responsibilities without extraneous elements
- Low **coupling** between classes, or very limited dependencies from one class to another

Five principles of OOP—the **SOLID Principles**—help deliver high cohesion and low coupling:

- **Single Responsibility Principle:** A class should only have one reason to change.
- **Open-Closed Principle:** Classes should be open for extension but closed for modification.
- **Liskov Substitution Principle:** Subtypes should be substitutable for their supertypes.
- **Interface Segregation Principle:** Classes shouldn't be forced to depend on interfaces they don't use.
- **Dependency Inversion Principle:** Depend on abstractions, not concretions.

OOP Patterns

Pattern: A named and well known problem/solution pair that can be applied in new contexts.

Examples of OOP patterns include:

- **Abstract Factory:** A creational pattern that provides an interface for creating objects without having to depend on the concrete classes that create them.
- **Adapter:** A structural pattern that allows use of instances through a different public interface than their class exposes.
- **Strategy:** A behavioral pattern that allows for the use of different algorithms in a class without having to hard code the algorithms in the classes themselves.
- **Observer:** A behavioral pattern that allows for an object to notify dependent objects of a change in state.

Note: The seminal work on OOP patterns is *Design Patterns: Elements of Reusable Object-Oriented Software* by the so-called “Gang of Four.”

Tip: The “Gang of Four” recommend you favor composition over inheritance when striving for code reuse.

THE FUNCTIONAL PROGRAMMING PARADIGM

Functional Programming

Functional programming (FP): Programs are built of individual functions that are assembled and applied to data. Data and the functions that operate on data are separate.

Characteristics of FP languages include:

- **First-class functions:** Functions can be treated as values.
- **Higher-order functions:** Functions can accept other functions as arguments and can return functions.
- **Declarative:** Code by describing the desired result (as opposed to specifying the steps to take).
- **Immutable data:** Data is unchangeable once defined.

FP Principles

To gain the benefits of FP, focus on these five principles:

- **Purity:** Functions must return a result based solely on their inputs, and must not rely on or alter external state.
- **Totality:** Functions must have a valid return value for every possible input.
- **Composition:** Functions must be small and focused, and designed to be able to be combined with other functions to achieve complex results.
- **Laziness:** Functions should avoid processing data until another function needs the data.
- **Recursion:** Functions should divide a multi-step task by calling itself with successively smaller parts of the task.

Note: Applying the five principles helps keep functions testable and supports parallelization of programs.

FP Patterns

The core pattern in FP is:

- **Filter:** Select data from a larger data set to work on.
- **Map:** Invoke functions on the data to get new data.
- **Reduce:** Invoke functions on multiple elements to get a single result.

Currying: Converts a function that takes multiple arguments into a function that takes one argument. It returns a function that takes the second argument, which in turn returns a function that takes the third argument, and so on.

Partial application: Involves calling a curried function with some of its arguments, getting a function that takes the remaining arguments.

Tip: Use currying and partial application when mapping and reduction functions take more than one argument.

FP Contexts

In FP, values are just data. When you need to convey something about the data, wrap it in a **computational context**, which contains additional information that accompanies the value and/or the interrelated conditions that define how a value can be treated.

Examples of computational contexts:

- **Tuple:** These values are an ordered, immutable collection that can be iterated over. Individual values can be selected without selecting others.
- **Maybe:** This is either a value or nothing (perhaps because the value comes from a function that can fail).
- **IO:** This value came from an I/O action, and the context contains the action that created it.
- **Either:** This value can be treated in one of two ways (perhaps as a result of a successful function execution or a message describing an error condition).

Advanced FP Data Types

There are three advanced data types that can be applied to ensure functions that operate on values in context preserve the context.

- **Functor:** A function that maps a context-unaware function to a value in a context to get another value in that same context.
- **Applicative:** A functor that maps a function in a context to a value in that context to get another value in that same context.
- **Monad:** An applicative that uses a value in a context as input to a function that takes a context-free input and returns a value in the context.

A Closer Look:

- Use a functor when filtering or mapping values in a context.
- Use an applicative when partially applying functions with values in a context.
- Use a monad to chain a series of operations together (such as in a reduction).

ENTERPRISE ARCHITECTURE AND BUSINESS LOGIC

Domain-Driven Design

Domain-Driven Design (DDD): A software engineering approach that aims to model how businesses work.

DDD ultimately represents the business environment as a collection of **conceptual** classes that each represent some real-world entity.

Domain experts: Subject matter experts in the business domain who provide the business expertise that engineers ultimately transform into software requirements.

Business domain: The value-generating service a company offers to its customers.

Subdomains: Discrete business areas within a business domain that work together to fulfill a business's goals.

Core subdomains: Represent a company's value to customers, and can be considered its reason for being.

Generic subdomains: Facilitate business without being a source of competitive advantage.

Supporting subdomains: Support a core subdomain.

Ubiquitous language: A common vocabulary agreed upon by stakeholders, which leads to a shared understanding across teams, reducing confusion and slowdowns.

Even with a ubiquitous language, some concepts may have different meanings depending on where they're used. For that reason, designers create **bounded contexts** in their models to set boundaries within which the meaning of a term is agreed upon.

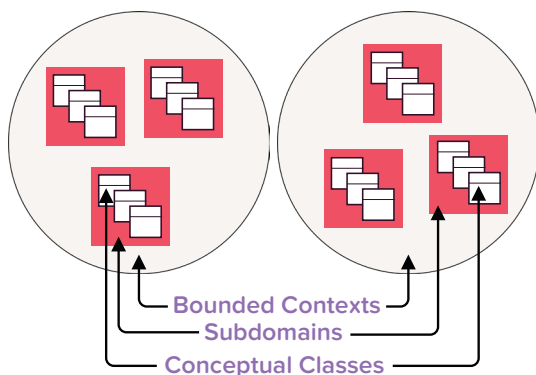
Business-domain model: A visual map illustrating key concepts within the domain.

The business-domain model reflects all relevant business units and shows cause-and-effect relationships.

Important: Software engineers place their greatest emphasis on developing core subdomains into successful systems.

Remember: The activities of generic subdomains are done largely the same way across many industries. Because of that, their development is often outsourced.

A Closer Look: Like generic subdomains, supporting subdomains don't provide any significant competitive edge. Unlike generic subdomains, they require specialization and differentiation.



Software Architecture

Software architecture: The organization of components in a software system.

Modules: Divide the total functionality of the system into relatively independent parts.

Patterns offer ways to ensure **separation of concerns** between modules, minimizing linkages and allowing for more freedom in changing any particular module.

Antipatterns: Solutions in principle that cause more harm than good in practice.

Orthogonality: The extent to which each module's responsibilities and functions are exclusive to that module.

Types of Patterns

- **Architecture:** How software is structured at the most general level, which informs how modules are organized and connected.
- **Business logic:** How real-world business rules are implemented via data creation, processing, and storage.
- **Data source:** How persistent data—e.g., in a database—is represented in business logic.
- **Presentation:** How the user interface is generated and connected to the application.

Remember: A system's architecture should be geared to meet its particular needs, and no single architecture is ideal for every system.

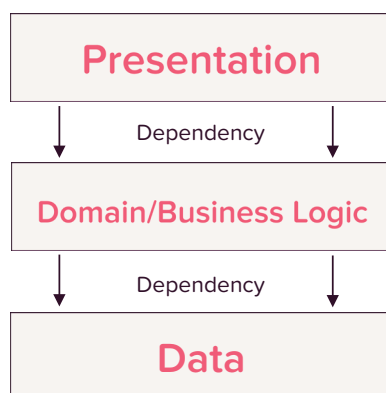
A Closer Look: There are two basic approaches to software architecture design.

- **Top down:** Scoping the whole, then gradually deconstructing it into more fundamental components.
- **Bottom up:** Specifying the most fundamental components, then building up and out.

Caveat: When misapplied, even the most useful patterns can become antipatterns.

Layered Architecture Patterns

Layers: An architecture pattern in which a system's modules are arranged as discrete layers in a vertical stack.



Presentation layer: Concerns how users interact with the system, which usually involves a graphical user interface (GUI) like a web browser or a smartphone display.

Business-logic layer: Where data is processed according to the business rules discovered during DDD.

Data layer: Manages the storage and retrieval of information processed by the application.

Tiers: The physical locations the software runs on.

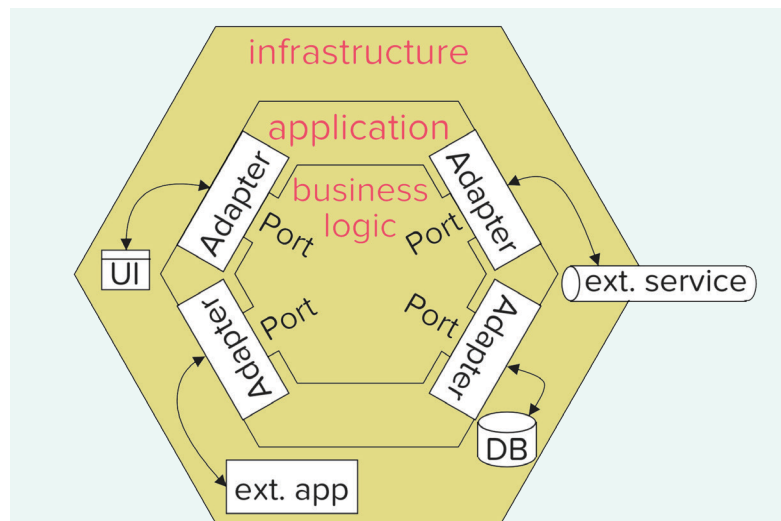
Service layer: Lies between the presentation and business-logic layers, effectively decoupling them.

The service layer orchestrates calls to the business logic so the presentation layer doesn't have to know about business logic, allowing business logic to change without changing presentation

Ports & Adapters: An architecture that makes everything else in the system dependent on the business-logic layer.

Caveat: Sometimes people conflate layers and tiers, but they're different concepts. Layers are simply how the software is organized into modules.

Important: Adapters allow business logic and infrastructure to vary independently.



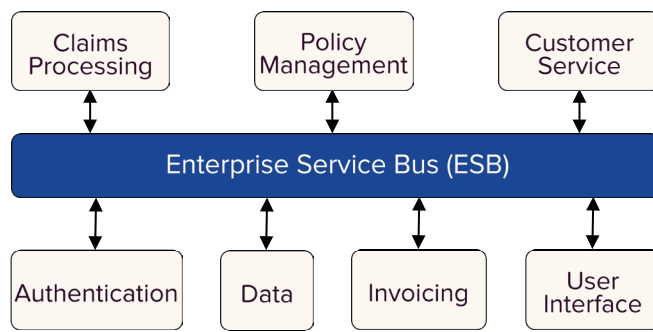
In the Ports & Adapters pattern, ports, which are defined in the business logic, manage incoming and outgoing data. Adapters, created in the application layer, connect elements like external services to the ports.

Service-Based Architecture Patterns

Service-Oriented Architecture (SOA): Decomposes a system into several smaller services that work together. This addresses issues typical of monoliths, including limited scalability, high coupling, and cumbersome integration and deployment.

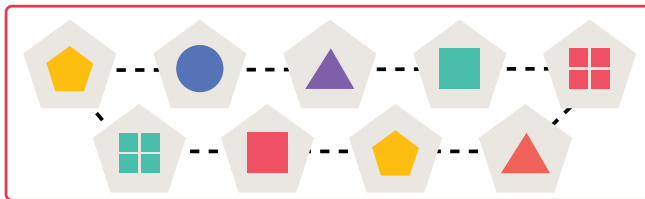
Services: Modules within SOA that contain all the code and data needed to perform a distinct business function.

Enterprise service bus (ESB): A standard feature of SOA that's responsible for service integration.



Microservices: An approach that breaks down an app's functionality into several services that can be developed and scaled independently.

Microservices app



Microservices vs. SOA

	Microservices	SOA
Share resources	Not generally	Yes
Reuse services	No	Yes

A Closer Look: An SOA service consists of three parts.

1. The contract, or the service's obligations to its consumers and vice versa
2. The interface, or the means by which the service's consumers make requests
3. The implementation, or the service's code

Business-Logic Patterns

Transactions: Discrete interactions or exchanges involving multiple steps. Transactions are atomic; if one part fails, the system must roll back to the pre-transaction state.

Transaction Script: A pattern that makes each transaction into a separate script or procedure.

Procedure: A series of statements or commands that orchestrates a transaction, rolls back in the event of failure, and guarantees atomicity.

Domain Model: A pattern that uses conceptual classes as the templates for creating domain objects.

Domain object: Represents an instance of some business entity.

Domain objects can exist within aggregates, or assemblies of domain objects treated as a whole.

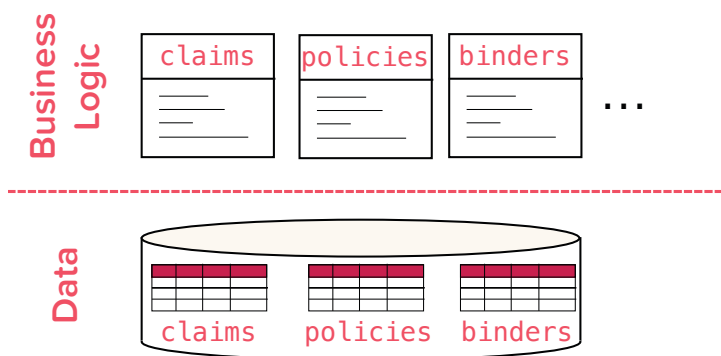
Aggregate root: The main entry point for accessing and modifying data in the aggregate.

Table Module: A pattern that creates a 1:1 association between business logic and the tables in the database.

Each table is represented by one class in the business-logic layer that can modify the data in that table, and each of those classes has just one instance.

Tip: Business logic is tied to the domain object, so it doesn't need to be duplicated when the domain object is used in different transactions.

Important: Other parts of the application can interact with the aggregate only through the aggregate root. This ensures data consistency, prevents unintended changes, and guarantees the atomicity of transactions.



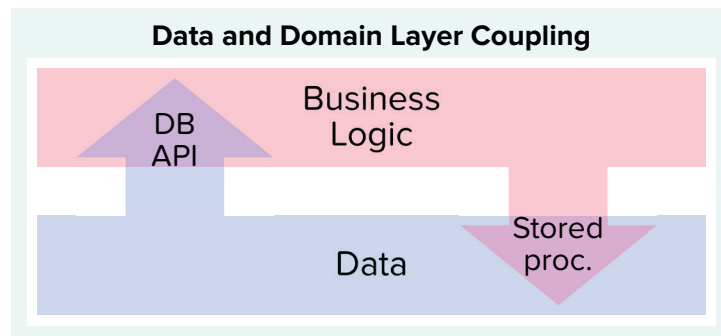
ENTERPRISE APPLICATION INFRASTRUCTURE

Working with Databases

Impedance mismatch: The set of inherent differences in the way data is represented between the business logic, which is often an object-oriented program, and the data store, which is an SQL-based relational database.

There are two basic ways to remove SQL from the business logic, but they both pose problems with maintainability, scalability, and high coupling.

1. **Stored procedures:** Pre-written SQL statements stored in the database that business logic can invoke with parameters.
2. **Database connectivity APIs:** Decouple the specific database platform from application code that accesses it.



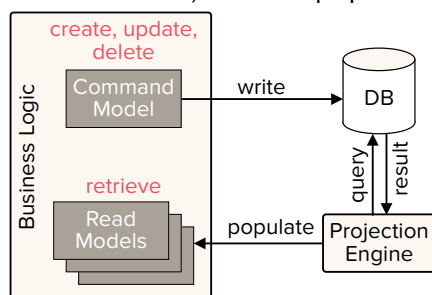
3.

An **object-relational mapping (ORM)** tool represents the database via objects and handles translations entailed by impedance mismatch.

Command-Query Responsibility Segregation (CQRS): A pattern that splits operations that read data from those that write it.

In CQRS:

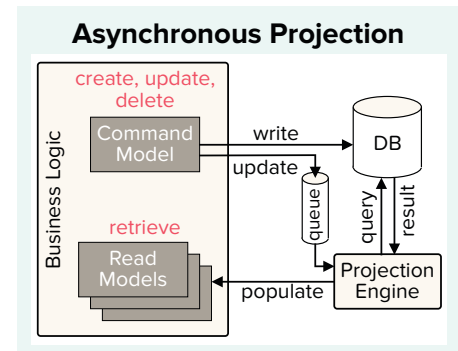
- All changes to the database are made by the command model in the business logic.
- All readable data is retrieved and stored in one or more read models, which is populated by a projection engine.



Note: ORM is a better solution to impedance mismatch than stored procedures or database connectivity APIs.

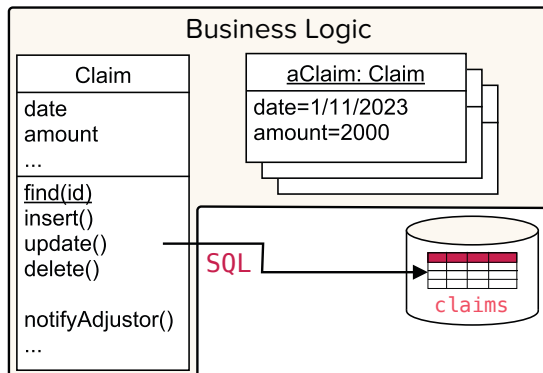
There are two models for consistency in a distributed data store (i.e., the database and projection engine).

- To get strong consistency, data should be kept consistent across multiple elements. CQRS achieves strong consistency via **synchronous projection**, a timestamp-based solution.
- Eventual consistency: An approach in which different elements in a distributed data store are eventually updated with new data. In CQRS, eventual consistency is implemented via **asynchronous projection**, which sends updates directly to the projection engine in addition to writing to the database.



Data-Source Patterns

Active Record: The simplest data-source pattern in which rows from the database are represented as domain objects in business logic.



Row Data Gateway: A pattern in which the gateway is a class whose instances each represent a single row in a database table.

The Row Data Gateway pattern has methods to create, update, and delete information, along with a finder class that has methods to read data from the database.

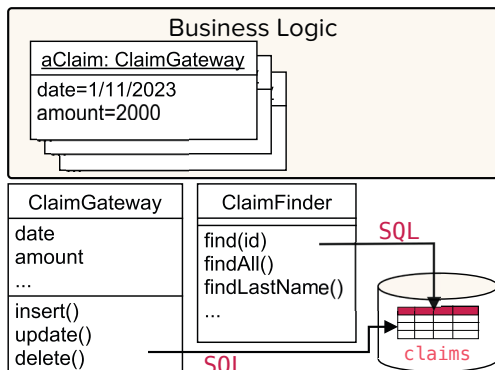
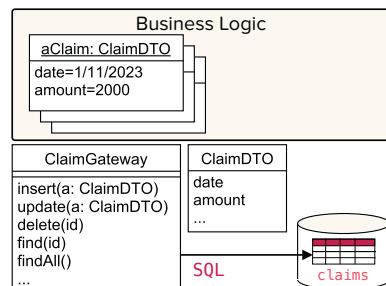


Table Data Gateway: A pattern that uses a single instance of a gateway class to access the table.

- The gateway class has methods to both read and update the table.
- Table rows are represented by **data transfer objects (DTOs)**, which only have attributes.
- To update the database, business logic passes the entire DTO to the gateway instance.



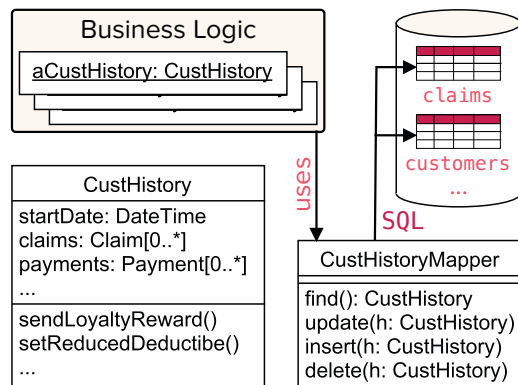
Data-Source & Business-Logic Pattern Pairings

Data-Source Pattern	Business-Logic Pattern(s)	Details
Active Record	Domain Model or Transaction Script	These patterns work well together if the domain objects are closely aligned to the database structure and their relationships aren't too complex.
Row Data Gateway	Transaction Script	These patterns work well together when the business logic involves relatively few transactions that are less complex.
Table Data Gateway	Transaction Script or Table Module	These patterns work well together because, in Transaction Script, procedures invoke methods in the gateway instance and work with the DTOs they provide. In Table Module, the business logic corresponds closely with the database table layout.

Advanced Data-Source Patterns

Data Mapper: A pattern that removes the business logic's dependency on the database schema using a layer of software called a mapper class.

The mapper class lies between the business logic and the database.



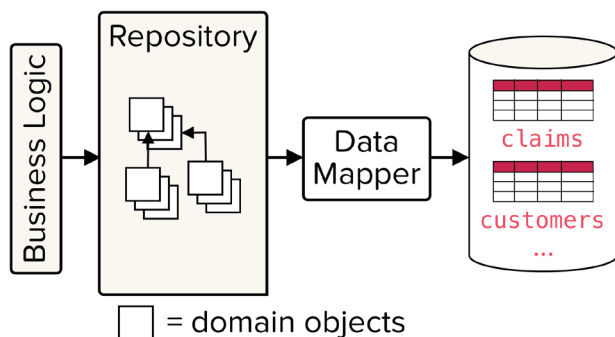
Important: Data Mapper replaces the responsibilities of Row and Table Data Gateway for database access and object instantiation. In addition, code in the mapper translates between the structure of the business logic and the schema of the database.

Persistence ignorance: The ability to completely isolate the business logic from database persistence.

Repository: A pattern that helps achieve persistence ignorance by making persistent data appear to business logic like an in-memory collection of objects and completely abstracting away the data layer..

Complex queries are executed by the repository instead of in business logic or in a data mapper.

Note: You should consider complex patterns like Data Mapper and Repository only when the complexity of your business logic calls for them.



Deconfliction and Synchronization

Identity Map: A pattern that keeps a record of the domain objects that have been read from the database for a given transaction.

By keeping each and every object loaded in a map, the identity map pattern ensures that no object is loaded more than once.

Object graphs: Groups of objects that are connected to each other.

Eager loading: Querying a database for all the data needed to build the object graph.

Lazy loading: A strategy where only a small amount of memory is used at any given time because only domain objects that are actually needed are loaded into memory.

Unit of Work (UoW): A pattern that tracks all changes that occur in the business logic.

When a business logic transaction is successfully completed, UoW builds a database transaction to persist the results.

UoW controller: Compares the state of objects before and after a business transaction, and then issues all the change registration calls to the appropriate UoW objects.

Event-Driven Architecture

Messages: Data objects with information from a sender.

Event: A message that describes something that happened in the domain.

Events are posted to a **message bus**, which is responsible for calling one or more **handlers** that take appropriate action when particular events happen.

Event-Driven Architecture (EDA): A pattern that involves modules posting messages for receivers to act on, as opposed to calls between modules.

To maintain aggregate state in a full event-driven architecture, aggregates raise domain events for all their state changes and then persist them to an **event store**.

Consistency boundary: A contained set of logic that must either entirely succeed or completely fail.

Since events are meant to be handled outside the domain, they're outside the consistency boundary of a transaction and should be handled after the transaction is committed.

Command: A message requesting that the system perform a task and provide feedback on its success.

The feedback that goes back to the aggregate that issued the command should be handled as a separate transaction.

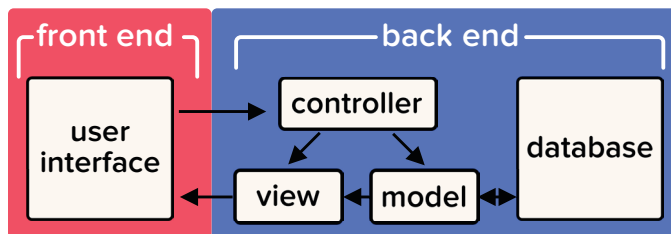
Event Sourcing pattern: Recreates the state of the aggregates at any point in their life by replaying the events that changed their state (in order, from first to last).

Presentation Patterns

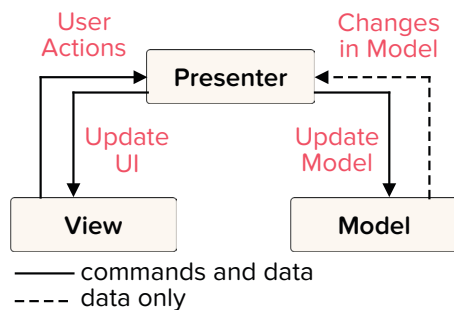
Model-View-Controller (MVC): A pattern that splits responsibilities for the user interface into three distinct roles.

There are two types of MVC controllers.

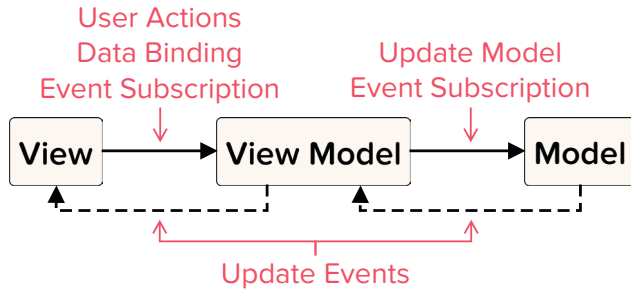
- A **page controller** is an object that handles requests for a specific page.
- A **front controller** handles requests for all the pages in an application using a single object.



Model-View-Presenter (MVP): A pattern that renames the controller to the presenter and makes it an intermediary between the view and the model, shifting the view's dependency away from the model.



Model-View-ViewModel: A pattern that eliminates two-way dependencies and uses binding and events to enable automatic updates to the view.



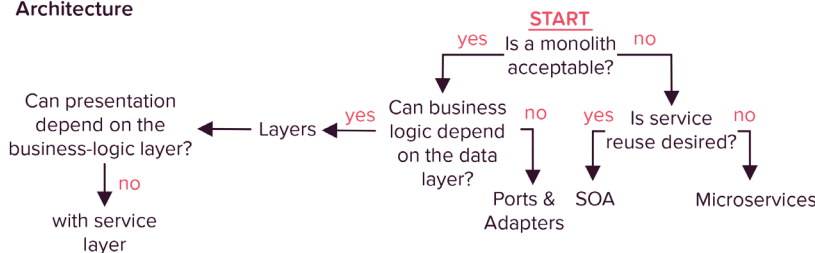
Remember: As you shift more responsibility for generating HTML from the server to the client, you move to SPA.

Single-Page Application (SPA): A model that loads just one web document for a given page and then updates using Ajax.

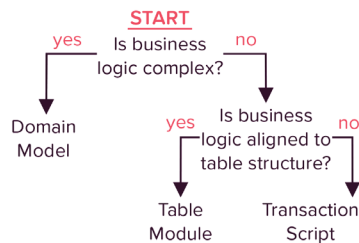
Choosing and Combining Patterns

Heuristic: A framework for making decisions that, while not always perfect, can help you identify solutions to problems.

Architecture



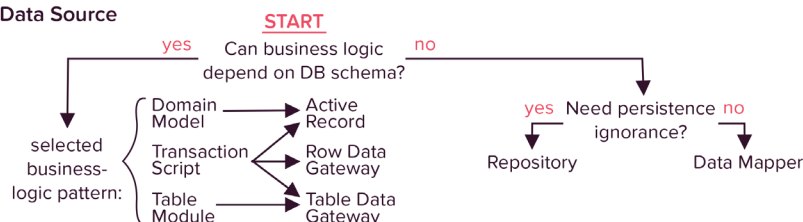
Business Logic



Presentation



Data Source



CLOUD SERVICES AND ARCHITECTURES

Cloud Services: Compute, Storage, and Database

Cloud services: Resources for hosting apps in the cloud.

Using a cloud service is called **provisioning** it.

Compute services: Resources that process and act on data.

Depending on your architecture and demand, you could have just one or even thousands of compute services running.

Storage services: Allow your app to store and retrieve file- and object-based data.

Virtual disk: The most basic storage service, which has a fixed size and can be attached to a virtual machine (VM) to serve as one of its drives.

Shared file system: Allows multiple compute services (including VMs, containers, and FaaS functions) to share a set of files organized in folders.

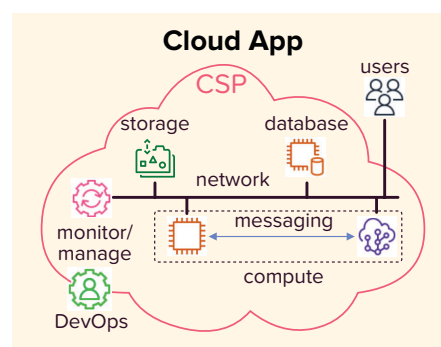
Unlike virtual disks, shared file systems grow and shrink as data is stored and removed.

Object storage: Allows you to store arbitrary amounts of data and retrieve that data using a name you assign when you store it.

Like shared file systems, object storage grows and shrinks as data is added and removed.

Database services: Services offered by CSPs that provide database storage, management, and administration.

Note: The “big three” CSPs are Amazon Web Services (AWS), Microsoft Azure, and Google Cloud.



Remember: Different types of compute services offered by cloud service providers (CSPs) include SaaS, PaaS, FaaS, and IaaS.

Storage Services

	virtual disk	shared file system	object storage
size	fixed	variable	variable
shared	limited	yes	yes
organized using	folders	folders	names
accessed by	VMs	cloud services	cloud & external services

Cloud Services: Networking, Messaging, Monitoring, and Management

Network services: Support connections between individual cloud services running in your cloud infrastructure and connections to the wider internet.

Virtual network: Connects cloud services to each other and the internet.

Load balancer: Distributes incoming traffic to multiple servers so they can share the work of processing requests.

Messaging services: Services offered by CSPs that allow cloud services to publish messages that other cloud services can subscribe to.

There are two basic messaging service models.

Services subscribe to a **topic** and publishers post messages to that topic like a bulletin board. Each subscriber receives every message the publisher publishes.

Services read from a **queue** that publishers add messages to. Once a service has read a message, that message is removed from the queue. Each message goes to only one service.

Monitoring and management services: Collect metrics on cloud services and take action based on set thresholds.

Remember: Load balancers help achieve horizontal scaling and monitor the health of the servers they work with.

Enterprise Architecture in the Cloud

Integration platforms as a service (iPaaS): Cloud-based app integration services, which take the place of enterprise service buses, service layers, and/or adapters in enterprise architectures.

Benefits of cloud computing include:

- Improved reliability
- Greater scalability
- Better return on investment (ROI)
- Faster time to market

Infrastructure as code (IaC): Capabilities offered by CSPs, which use descriptive files or imperative programs to automate the configuration and provisioning of services.

There are two basic approaches to IaC.

1. Declarative, in which one or more files describe the needed configuration of services, and a cloud service builds the infrastructure.
2. Imperative, in which a program executes a series of commands that configure and provision services.